

# CSCI 232

## 232 Algorithms & Data Structures

**LECTURE: 5**

J. L. Pach

# OUTLINE:

- Unit test
- Engine Unity for C/C++
- Review of C/C++
  - Relations between pointers and arrays

# UNIT TESTS

# WORKING WITH CODE

https://github.com/jpach-cs/C_tests

# WHAT ARE UNIT TESTS

Unit tests are automated checks of small parts of a program (such as functions or procedures) to verify that they work correctly. The idea is to test whether a given function returns the correct result for specific input values.

- Why use unit tests?
  - You catch bugs faster.
  - You don’t have to manually test your code every time.
  - You gain confidence that changes in the code don’t break other parts (this is called regression prevention).

# INTRODUCTION TO UNITY TEST IN C

When we write programs, we need a way to check if our code works correctly. There are two common tools for this in C:

- Assertions (assert)
- Unit testing frameworks (like Unity Test)

# WHAT IS AN ASSERT?

- Assert() is a built-in tool in C. (#include <assert.h>)
- It is mainly used by the programmer while writing code to check if assumptions are correct.
- If the condition in assert is false, the program immediately stops (crashes). Example:
- Good for debugging, **but not suitable for automated testing**, because once it fails, the program cannot continue.

```c
#include <assert.h>
int divide(int a, int b)
{
    assert(b != 0);  // program will stop if b == 0
    return a / b;
}
```



# WORD – ASSERT

- In structurally or imperatively oriented programming, function names are typically nouns — for example, `sum()`, `pow()`, or `strlen()`.
- In object-oriented programming, we create instances of objects represented by nouns, but the methods invoked on those objects are usually verbs, describing actions performed on the instance — for example, `trash.clean()`.
- In the C language, the function `assert()` is an exception. Unlike typical function names, it is a verb, reflecting the programmer's intention to assert — to strongly claim — that a certain condition holds true.

Conceptually, it's as if the programmer is saying: "I assert that x equals 1" `assert(x==1);`

# WORD – 'ASSERT'

- The original intent behind `assert()` was to allow developers to write code like:
  - `sum(a, b);`
  - `assert(a > 0);`
- These statements were meant to halt program execution if the condition was not met. In short, they served as additional safeguards to help speed up debugging.
- Over time, **most programming languages adopted dedicated unit testing frameworks** — even C, thanks to the Unity library. Unity was designed with **a plan-driven** approach in mind, enabling developers to write tests before implementing the actual code.

# EXIT CODE / RETURN CODE / STATUS CODE

- The program returned exit code `0`, which means it ran successfully.
- A non-zero exit code usually indicates an error or failure.

```c
int main(int argc, char *argv[])
{
    return 0; // cmd/powershell:  echo $LASTEXITCODE
}
```



```
C:>main.exe
C:>echo $LASTEXITCODE
0
C:>
```

# WHAT IS UNITY TEST?

- Unity Test is a unit testing framework for C.
- It allows us to test functions in a safe and controlled way.
- Unlike assert, Unity Test does not stop the program when a test fails. Instead, it records the failure and continues with other tests.
- This way we get a summary of all passed and failed tests at the end. Example test with Unity:

```c
#include "unity.h"
void test_addition(void)
{
    TEST_ASSERT_EQUAL(4, 2 + 2);  // this will pass
    TEST_ASSERT_EQUAL(5, 2 + 2);  // this will fail, but program continues
}
```



# THE DIFFERENCE: ASSERT VS. UNITY TEST

- `Assert()` → for the programmer, during development, to catch bugs early.
- Unity Test → for systematic testing of finished functions, with clear reports.

# Error handling in C

- C has no exceptions (like in Java or Python).
- Instead, functions use special return values to signal errors:
  - Functions returning int often use -1 to mean *something went wrong*.
  - Functions returning pointers use `NULL` to mean *failure*.
  - At the program level, the exit code follows the same rule:
    - `0` = success
    - `non-zero` = error

```c
int findElement(int arr[], int size, int target)
{
    for(int i = 0; i < size; i++)
    {
        if(arr[i] == target)
            return i;  // found, return index
    }
    return -1; // not found → error
```



# SUMMARY

- Use assert inside your code while developing → catches programmer mistakes early.
- Use Unity Test to run proper unit tests on your functions.
- Handle errors with special return values (`-1` for `int`, `NULL` for pointers), so tests can check failures without crashing the program.

# INTRODUCTION TO UNITY TEST FRAMEWORK IN C

Unity is a lightweight testing framework for the C language. It allows us to write **unit tests** that check whether our functions work as expected.

When using Unity, every test file usually has three important parts:

- `setUp()` – a function that runs before each test. You can use it to prepare data or reset variables.
- `tearDown()` – a function that runs after each test. You can use it to clean up resources.
- `UNITY_BEGIN()`; `RUN_TEST(...)`; **return** `UNITY_END()`; – this is the main pattern for starting Unity, running all tests, and reporting the results.

# Common Unity Assertions

Assertions are the heart of testing. They compare the expected result with the actual result and tell us if the test passed or failed. Some of the most common are:

- `TEST_ASSERT_EQUAL(expected, actual)`
  - check if two integers are the same.
- `TEST_ASSERT_NULL(ptr)` / `TEST_ASSERT_NOT_NULL(ptr)`
  - check if a pointer is NULL or not.
- `TEST_ASSERT_TRUE(condition)` / `TEST_ASSERT_FALSE(condition)`
  - check if a condition is true or false.
- `TEST_ASSERT_FLOAT_WITHIN(delta, expected, actual)`
  - check if two floating-point numbers are equal within a tolerance.
- `TEST_ASSERT_EQUAL_STRING(expected, actual)`
  - check if two strings are equal.

# COMMON UNITY ASSERTIONS

- `TEST_ASSERT_EQUAL(expected, actual)`
  - check if two integers are the same.
- `TEST_ASSERT_NULL(ptr)` / `TEST_ASSERT_NOT_NULL(ptr)`
  - check if a pointer is NULL or not.
- `TEST_ASSERT_TRUE(condition)` / `TEST_ASSERT_FALSE(condition)`
  - check if a condition is true or false.

# COMMON UNITY ASSERTIONS

- `TEST_ASSERT_FLOAT_WITHIN(delta, expected, actual)`
  - check if two floating-point numbers are equal within a tolerance.
- `TEST_ASSERT_EQUAL_STRING(expected, actual)`
  - check if two strings are equal.
- `TEST_ASSERT_GREATER_THAN(threshold, actual)`
  - custom/extended check if a value is greater than a threshold.

# UNDERSTANDING `setUp()` AND `tearDown()` 1/2

In Unity, the function `setUp()` is always called before each test. You use it to prepare the environment: initialize variables, reset arrays, or allocate memory.

The function `tearDown()` is always called after each test. At first, this may look unnecessary. In modern languages with a garbage collector (like Java or Python), developers rarely think about memory allocation and cleanup, so they might wonder: *Why do I need `tearDown()` if `setUp()` can just reset the data again?*

# UNDERSTANDING `setUp()` AND `tearDown()` 2/2

The answer is that in C and C++ we often use dynamic memory. If a test uses `malloc()` (or new in C++), we must free that memory after the test, otherwise we create memory leaks. That's why `tearDown()` is important—it ensures that all resources used by a test are properly released, no matter if the test passed or failed.

So the pattern is:

- `setUp()` → prepare everything a test needs.
- `tearDown()` → clean up, especially for dynamic memory.

This makes every test independent, safe, and repeatable.

# FUNCTION

## arguments

# FUNCTION ARGUMENTS

Function arguments are always **copies** of our variables, and **not** the same memory areas, a function argument, even though it has the same value, is a completely different variable!

# by the Value

Function arguments are always **copies** of our variables, and **not** the same memory areas, a function argument, even though it has the same value, is a completely different variable!

```c
#include <stdio.h>

void byTheValue(int);
void byTheValue(int value)
{
    value++;
    return;
}
int main()
{
    int x = 5;
    printf("%d\n", x);          // 5
    byTheValue(x);
    printf("%d\n", x);          // 5
  return 0;
}
```

# by the Reference

Function arguments are always **copies** of our variables, and **not** the same memory areas, a function argument, even though it has the same value, is a completely different variable!



```c
#include <stdio.h>

void byTheReference(int*);
void byTheReference(int* ref)
{
    (*ref)++;
    return;
}
int main()
{
    int x = 5;
    printf("%d\n", x);              // 5
    byTheReference(&x);
    printf("%d\n", x);              //6
  return 0;
}
```

# Some examples

```c
int main()
{
  int x = 5, y = 7;
  int * p = &x;
  int ** pp = &p;   /* pointer to pointer*/
  y = **pp;
  printf("y equals %d.\n\n",  y );

  printf("&y\t\tequals %d.\n", &y );
  printf("*(&y)\t\tequals %d.\n", *(&y) );
  printf("&(*(&y))\tequals %d.\n", &(*(&y)) );
  printf("&*&y\t\tequals %d.\n", &*&y );
  printf("*&*&y\t\tequals %d.\n\n", *&*&y );

  printf("&x\t\tequals %d.\n", &x );
  printf("p\t\tequals %d.\n", p );
  printf("&p\t\tequals %d.\n", &p );
  printf("*p\t\tequals %d.\n\n", *p );

  printf("pp\t\tequals %d.\n", pp );
  printf("&pp\t\tequals %d.\n", &pp );
  printf("*pp\t\tequals %d.\n", *pp );
  printf("*&*pp\t\tequals %d.\n", *&*pp );
  printf("**pp\t\tequals %d.\n", **pp );
  return 0;
}
```

# Some examples

Result:

```
y equals 5.

&y              equals 6422292.
*(&y)           equals 7.
&(*(&y))        equals 6422292.
&*&y            equals 6422292.
*&*&y           equals 7.

&x              equals 6422296.
p               equals 6422296.
&p              equals 6422288.
*p              equals 5.

pp              equals 6422288.
&pp             equals 6422284.
*pp             equals 6422296.
*&*pp           equals 6422296.
**pp            equals 5.
```

# Some examples

| Memory Addresses and Values | |
|---|---|
| x (6422296) | 5 |
| y (6422292) | 5 |
| p (6422288) | 6422296 |
| pp (6422284) | 6422288 |

# RELATIONS

## between pointers and arrays

# RELATIONS BETWEEN POINTERS AND ARRAYS

This statement reserves space in memory for 10 integers and creates an

> *unchanging address of memory*

that points to the beginning of this array.

```c
int a[10];
```

```c
int main()
{
  int a[] = {1, 2, 3, 4, 5, 6, 7};
  printf("%-4s equals %d.\n", "a", a);       // a    equals 6422272.
  /*a equals'unchanging address of memory'*/
  return 0;
}
```

# RELATIONS BETWEEN POINTERS AND ARRAYS

```c
int main()
{
  int a[] = {1, 2, 3, 4, 5, 6, 7};
  printf("%-4s equals %d.\n","a", a);
  printf("%-4s equals %d.\n","&a", &a);
  printf("%-4s equals %d.\n","*a", *a);
  return 0;
}
```



Result:

```
a    equals 6422272.
?
```

# RELATIONS BETWEEN POINTERS AND ARRAYS

```c
int main()
{
  int a[] = {1, 2, 3, 4, 5, 6, 7};
  printf("%-4s equals %d.\n","a", a);
  printf("%-4s equals %d.\n","&a", &a);
  printf("%-4s equals %d.\n","*a", *a);
  return 0;
}
```



Result:

```
a    equals 6422272.
&a   equals 6422272.
?
```

# RELATIONS BETWEEN POINTERS AND ARRAYS

```c
int main()
{
  int a[] = {1, 2, 3, 4, 5, 6, 7};
  printf("%-4s equals %d.\n","a", a);
  printf("%-4s equals %d.\n","&a", &a);
  printf("%-4s equals %d.\n","*a", *a);
  return 0;
}
```



Result:

```
a    equals 6422272.
&a   equals 6422272.
*a   equals 1.
```

# RELATIONS BETWEEN POINTERS AND ARRAYS

## So, what does it mean?

```c
int main()
{
  int x = 5, y = 7, z = 9;
  int * pointer = &y;
  printf("%-4s equals %d.\n", "x", &x);  printf("%-4s equals %d.\n", "y", &y);  printf("%-4s equals %d.\n", "z", &z);
  printf("%-4s equals %d.\n", "pointer", pointer);
  printf("%-4s equals %d.\n", "pointer*", *pointer);
  printf("%-4s equals %d.\n", "pointer[1]", pointer[1]);
  return 0;
}
```

```
x    equals 6422292.
y    equals 6422288.
z    equals 6422284.
pointer equals 6422288.
pointer* equals 7
```

# SO, WHAT DOES IT MEAN?

When performing arithmetic operations on pointers, the array index is automatically multiplied by the size of the data type pointed to by the pointer.

For example, if the array stores `char` values, the index is multiplied by 1, and if it stores int values, the index is multiplied by 4 (assuming an `int` is 4 bytes).

```
int a[10];

a[i] == *( a + i)          <==>          &a[i] == &*(a + i) == a + i
```



# CONCLUSIONS

- The expression `a[i]` is transformed by the compiler into the form `*(a+i)`.
- The square brackets following the `symbolic_name` do not provide any information about whether we are referring to an array.
- When performing arithmetic operations on pointers, the array index is automatically multiplied by the size of the data type pointed to by the pointer.

# crème de la crème

```c
int main()
{
    unsigned int x = 0x41424344;    /* Store a 32-bit integer with ASCII-like hex values */
    char *p = (char*) &x;           /* Cast the address of x to a char pointer */
    printf("x = 0x%X\n", x);                                    // x = 0x41424344
    /* Using p[i] does not mean we have an array. */
    /* In C, p[i] is just shorthand for *(p + i), */
    /* which means "take the value at address p plus i bytes." */
    /* Since p is a char*, each step moves by 1 byte. */
    /* The actual output depends on system endianness: */
    /* - Little-endian: p[0] = 'D', p[1] = 'C', p[2] = 'B', p[3] = 'A' */
    /* - Big-endian:    p[0] = 'A', p[1] = 'B', p[2] = 'C', p[3] = 'D' */
    printf("p[0] = %c\n", p[0]);                                    // p[0] = D
    printf("p[1] = %c\n", p[1]);                                    // p[1] = C
    printf("p[2] = %c\n", p[2]);                                    // p[2] = B
    printf("p[3] = %c\n", p[3]);                                    // p[3] = A
    return 0;
}
```

# THANK

**You**